

MESOS

FUNZIONALITÀ IMPLEMENTATE

GESTIONE SERVER E LOBBY

- Login dei giocatori
- Lobby da 2 a 5 giocatori
- Scelta del colore e ready check
- Gestione di ere, round e turni

MECCANICHE DI GIOCO

- Carte personaggio, edificio ed evento
- Bonus ed effetti permanenti e temporanei
- Costo in cibo degli edifici
- Sustenance e punteggio finale

FUNZIONALITÀ AGGIUNTIVE

- Timeout e riconnessione
- Sospensione con un solo giocatore online
- Salvataggio su disco
- Leaderboard JDBC opzionale

COMUNICAZIONE CLIENT-SERVER

- Client TUI e GUI JavaFX
- Socket e RMI nello stesso server
- Lobby con client misti
- Validazione lato server di ogni azione

DIVISIONE DEL LAVORO

TOTEM E MECCANICHE DI GIOCO — BENEDETTO OLIVA

- Gestione delle regole principali
- Piazzamento totem
- Calcolo del punteggio
- Carte ed effetti

RETE E INFRASTRUTTURA — GIORGIO ORTONA

- Socket e RMI
- Gestione messaggi
- Disconnessione e riconnessione
- Salvataggi

DOCUMENTAZIONE E ARCHITETTURA — ALESSANDRO MAZZEO

- UML e design pattern
- Testing e validazione
- Organizzazione del progetto
- Manuale e documentazione

INTERFACCIA UTENTE — CAMILLO NICOLETTI

- GUI JavaFX
- TUI
- Gestione eventi utente
- Esperienza di gioco

DESIGN PATTERNS UTILIZZATI

STATE

DOVE
LobbyState, TotemPlacementState, ActionResolutionState, SuspendedState, EndGameState

PERCHÉ LO USIAMO
Incapsula transizioni e invarianti di fase, evitando branch centralizzati e riducendo stati illegali.

COMMAND

DOVE
GameAction e sottoclassi

PERCHÉ LO USIAMO
Incapsula l'input utente in oggetti validabili prima dell'applicazione al dominio.

DTO / PROTOCOL MESSAGES

DOVE
serverbound, clientbound

PERCHÉ LO USIAMO
Definisce un boundary stabile di serializzazione; rete e dominio restano disaccoppiati.

FACTORY + STRATEGY

DOVE
ClientConnectionFactory, ClientSideConnection

PERCHÉ LO USIAMO
Applica dependency inversion: il controller dipende da un'interfaccia e il trasporto si seleziona a runtime.

ADAPTER

DOVE
SocketConnection, RMIConnection, VirtualView

PERCHÉ LO USIAMO
Normalizza API diverse, stream/RMI/callback, in un contratto comune isolando i dettagli infrastrutturali.

DISPATCHER

DOVE
MessageDispatcher, mappe handler nel client

PERCHÉ LO USIAMO
Usa una tabella di handler per tipo messaggio, eliminando catene instancocf e centralizzando il routing.

OBSERVER

DOVE
GameListener, callback setOnMessage, setOnDisconnect

PERCHÉ LO USIAMO
Realizza un publish/subscribe leggero: eventi verso view e rete senza dipendere dai subscriber concreti.

VISITOR

DOVE
carte ed effetti

PERCHÉ LO USIAMO
Usa double dispatch sugli effetti: comportamento specializzato senza switch sul tipo e con estensione localizzata.

SNAPSHOT

DOVE
GameStartedMessage, GameSyncMessage

PERCHÉ LO USIAMO
Trasferisce uno stato consistente e serializzabile, utile per bootstrap e recovery idempotente.

SUPERVISOR / RETRY

DOVE
ClientConnectionSupervisor

PERCHÉ LO USIAMO
Isola la politica di reconnsione con retry temporizzato, senza bloccare UI o duplicare logica di connessione.

PROTOCOLLO E PERSISTENZA

Il protocollo usa messaggi DTO serializzabili: sono oggetti semplici che trasportano dati tra client e server senza contenere logica di gioco. I ClientMessage includono LoginMessage, ReadyMessage, GameActionMessage, ReconnectMessage, DisconnectMessage, QuitMessage; i ServerMessage includono LoginAckMessage, GameStartedMessage, GameSyncMessage, StateChangedMessage, PlayerDisconnectedMessage, GameSuspendedMessage, GameResumedMessage.

Durante il gioco normale il server invia messaggi delta: solo ciò che cambia. Il GameSnapshot completo viene costruito solo quando serve una sincronizzazione completa: inizio partita o riconnessione. La persistenza non salva lo snapshot, ma serializza il GameController in mesos-savestate.bin; al restart il server ricarica il controller e genera un nuovo snapshot per ogni client che si riconnette.

MESSAGGI PRINCIPALI

DIREZIONE	MESSAGGI
CLIENT → SERVER	LoginMessage, ReadyMessage, GameActionMessage, ReconnectMessage, DisconnectMessage, QuitMessage
SERVER → CLIENT	LoginAckMessage, GameStartedMessage, GameSyncMessage, StateChangedMessage, PlayerDisconnectedMessage, GameSuspendedMessage, GameResumedMessage

FLUSSO RECONNECT

1. **GUASTO RILEVATO** — heartbeat, eccezione o onDisconnect
2. **CLEANUP SERVER** — LobbyManager pulisce la connessione
3. **STATO PARTITA** — disconnessione e sospensione
4. **RETRY CLIENT** — nuova richiesta ogni 2 sec
5. **RIALLINEAMENTO** — ack, snapsbot e stato corrente

TEST E COPERTURA

La suite contiene 97 classi di test su model, controller, stati, network, protocollo, persistenza e riconnessioni. La copertura è misurata con JaCoCo tramite mvn clean verify. GUI/TUI, launcher e JDBC leaderboard sono validati anche con prove manuali perché dipendono da interazione grafica, processo o database esterno.

COPERTURA ISTRUZIONI JACOCO

Globale	39.1%
Core senza view	75.8%
Model	90.9%
Controller	75.9%
Network	65.8%
Protocol DTO	88.9%

INDICATORI SINTETICI

Test classi	97
Branch globale	36.4%
Linee globali	42.0%
Core linee	74.3%
Comando coverage	mvn clean verify
Validazione manuale	GUI/TUI, launcher, JDBC